

推荐系统部分

矩阵分解 (一种协作过滤模型类别) 旨在解决用户-物品互动矩阵稀疏的问题, 把该矩阵分解为两个低秩矩阵的乘积

其中隐藏空间的维度 K 是用来描述用户偏好和物品类型偏好的隐藏因素的个数

举例: 比如 动作电影、爱情电影、混合电影, 那么 $K=2$, 表示用户偏好爱情和动作的占比, 还有各电影在这两个隐藏因素的占比, 比如用户 1 的用户矩阵 $P[1, 0]$, 指喜欢爱情, 完全不喜欢动作成份

- 注: P 是一个用户矩阵, P_u 才是用户 u 所对应的向量, Q 同样如此

然后把 P 与 Q 矩阵点积, 用户喜欢的属性与电影拥有的属性匹配度越高, 点积通常越大即预测评分越高

- 损失函数的计算: $L = \frac{1}{B} \sum_{j=1}^B (r_j - \hat{r}_j)^2$ 其中 B 为批量大小, 采用均方误差

- 由于用户-物品互动矩阵非常稀疏, 所以要正则化, 防止模型过拟合.

在 pytorch 中用 L2 正则化 则: $L = \sum_{(u,i) \in X} (R_{ui} - \hat{R}_{ui})^2 + \lambda (\|P\|_F^2 + \|Q\|_F^2 + \|b\|^2)$

可以采用优化器来实现: `optimizer = torch.optim.Adam(`

`model.parameters(),`

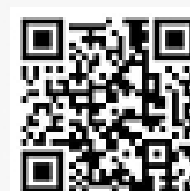
`lr = 0.02, // 学习率`

`weight-decay = 1e-5. => 会限制参数变得非常大`

- 注: 前面假定隐藏空间维度 K 以后, 每个用户向量和物品向量的初始值设定可以随机, 但在后续的训练时, 假定用户 5 给物品 2 打了 5 分即 $u_{5,2} = 5$, 则对应的 $P_5 \cdot Q_2$ 的结果应当尽可能的接近 5, 所以后续的梯度下降算法 T 逐步接近已有的准确值

显然在这种算法下每个用户向量会受到多个打分 ~~和~~ 的影响, 这样的结果再加上偏置泛化和正则化力就可以大大增强

- embedding 不能翻译成嵌入, 而应该为 latent features, 即隐藏特征 (潜在特征)



CS 扫描全能王

3亿人都在用的扫描App